MINISTRY OF EDUCATION AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics

REPORT

Laboratory Work No. 1

Topic: **Caesar Cipher**

The Caesar cipher and the Caesar cipher with a permutation for the Romanian alphabet

Author: Bîtca Cătălin
Group: FAF-242

Chișinău, 2026

1. Short theory

1.1. The Caesar cipher

The Caesar cipher is one of the oldest substitution ciphers. Every letter of the plaintext is replaced by the letter that stands a fixed number of positions k further in the alphabet. When we go past the last letter we continue from the beginning, so the alphabet works like a circle. If the letters are numbered from 0 to $n-1$, encryption and decryption are:

$$c = e_k(x) = (x + k) \bmod n, \quad m = d_k(y) = (y - k) \bmod n,$$

where x is the code of a plaintext letter, y is the code of a ciphertext letter and k is the secret key, which is the same for encryption and decryption. If $y - k$ is negative, n is added to bring the result back into the range $0 \ldots n-1$.

For this lab I used the Romanian alphabet, which has $n = 31$ letters. The codes of the letters are given in Table 1 and were used in both tasks.

Letter	A	Ă	Â	B	C	D	E	F	G	H	I	Î	J	K	L	M	N	O	P	Q	R	S	Ș	T	Ț	U	V	W	X	Y	Z
Code	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30

Table 1. *Letter encoding of the Romanian alphabet (n = 31)*

Key space. The key can only be 1, 2, …, $n-1$ ($k = 0$ leaves the text unchanged), so for the Romanian alphabet there are only **30** possible keys (25 for English). This is the main weakness of the cipher: an attacker can simply try every key (brute-force attack), and only one of them will give readable text.

1.2. The Caesar cipher with a permutation

To make the key space bigger, a second key is added: a **keyword** k_2. It is used to reorder the alphabet. First we write the letters of the keyword in the order they appear, keeping only the first occurrence of every letter, and after that the rest of the alphabet letters in their normal order. The Caesar shift with the numeric key k_1 is then done inside this new order, so the code of a letter becomes its position in the permuted alphabet. The formulas stay the same:

$$c = (x + k_1) \bmod n, \quad m = (y - k_1) \bmod n, \quad x, y = \text{positions in the permuted alphabet.}$$

Key space. There are 31! possible orderings of the Romanian alphabet and 30 possible shifts, so the number of key pairs (k_1, k_2) is

$$31! \times 30 = 246\,685\,159\,625\,337\,684\,531\,766\,886\,400\,000\,000 \approx 2.47 \cdot 10^{35}.$$

Checking all of them is not possible in practice. Still, the cipher remains monoalphabetic: each plaintext letter is always replaced by the same ciphertext letter, so the letter frequencies of the language stay visible and the cipher can be broken by **frequency analysis**. Also, different key pairs can give the same substitution, so the number of really different encryptions is at most 31!.

2. Implementation

I implemented both tasks in **Python 3**, in a single file `caesar.py`. When the program starts it shows a small menu: the user chooses Task 1.1 or Task 1.2, then the operation (encrypt or decrypt), then the key (and the keyword for Task 1.2) and finally the message or the ciphertext. After the result is printed the menu is shown again, until the user chooses 0 to exit. The encryption and decryption functions are the same for both tasks; the only difference is which alphabet is given to them: the normal one for Task 1.1 and the permuted one for Task 1.2. The most important parts of the code are shown below.

2.1. Letter encoding

```
import unicodedata

ALPHABET = "AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ"
LOWERCASE = "aăâbcdefghiîjklmnopqrsștțuvwxyz"
N = len(ALPHABET)
MIN_KEYWORD_LENGTH = 7
LETTERS = "Romanian letters (A–Z, Ă, Â, Î, Ș, Ț)"

UPPER_OF = {}
for i in range(N):
    UPPER_OF[ALPHABET[i]] = ALPHABET[i]
    UPPER_OF[LOWERCASE[i]] = ALPHABET[i]
UPPER_OF["Ş"] = "Ș"
UPPER_OF["ş"] = "Ș"
UPPER_OF["Ţ"] = "Ț"
UPPER_OF["ţ"] = "Ț"
```

The alphabet is stored as a string in exactly the same order as in Table 1, so the code of a letter is just its index in this string (A = 0, Ă = 1, …, Z = 30). Built-in encodings like `ord()` or Unicode code points are never used to compute the shift. `UPPER_OF` is a dictionary that maps every allowed character (uppercase or lowercase) to its uppercase letter. I built it from the two strings instead of using `str.upper()`, so that only letters of the Romanian alphabet are accepted. The last four lines add the letters Ş, ş, Ţ, ţ written with a **cedilla** (U+015E, U+015F, U+0162, U+0163) and map them to the correct Ș and Ț with a **comma below**, so text typed on different keyboards is treated the same way.

2.2. Preparing the text

```
def prepare_text(text):
    text = unicodedata.normalize("NFC", text)
    result = ""
    for ch in text:
        if ch == " ":
            continue
        if ch not in UPPER_OF:
            raise ValueError(ch)
        result += UPPER_OF[ch]
    return result
```

This function prepares the message (or ciphertext) before encryption or decryption. First it normalizes the string to Unicode NFC, so a letter like Ă typed as A + a combining breve becomes one character. Then it skips the spaces and converts each letter to uppercase using `UPPER_OF`. When it finds a character

that is not allowed, it raises `ValueError` with that character, so the input function can tell the user exactly which character was rejected.

2.3. Encryption and decryption

```
def encrypt(text, key, alphabet=ALPHABET):
    n = len(alphabet)
    result = ""
    for ch in text:
        x = alphabet.index(ch)
        result += alphabet[(x + key) % n]
    return result


def decrypt(text, key, alphabet=ALPHABET):
    n = len(alphabet)
    result = ""
    for ch in text:
        y = alphabet.index(ch)
        result += alphabet[(y - key + n) % n]
    return result
```

These two functions are a direct translation of the formulas from the theory. `alphabet.index(ch)` gives the code of the letter (x or y), and the result letter is taken from the same alphabet at the new position. In `decrypt` I add n before applying `%`. In Python `%` already returns a non-negative result, but this way the code does not depend on the language and works the same as in C or Java, where the remainder can be negative. Example with $k = 4$: Z (30) → (30 + 4) mod 31 = 3 = B, and back: (3 − 4 + 31) mod 31 = 30 = Z. The parameter `alphabet` is the normal alphabet by default, and for Task 1.2 the permuted one is passed.

2.4. Building the permuted alphabet

```
def permuted_alphabet(keyword, alphabet=ALPHABET):
    result = ""
    for ch in keyword + alphabet:
        if ch not in result:
            result += ch
    return result
```

The function goes through the keyword followed by the whole alphabet and adds a letter only if it is not already in the result. Because of this the keyword letters come first (only their first occurrence) and then all the remaining letters in their natural order, and the result always has exactly 31 letters. For the keyword **criptografie** (converted to CRIPTOGRAFIE) the second R and I are dropped and the permuted alphabet is:

C R I P T O G A F E Ă Â B D H Î J K L M N Q S Ș Ț U V W X Y Z

Table 2 shows this alphabet with the shift $k_1 = 5$, in the same format as the table from the lab description, so the result can be checked by hand. For example, S has position 22, so it becomes the letter at position 27, which is W; E has position 9 and becomes H (position 14).

x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30
m	C	R	I	P	T	O	G	A	F	E	Ă	Â	B	D	H	Î	J	K	L	M	N	Q	S	Ș	Ț	U	V	W	X	Y	Z
x+k	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	0	1	2	3	4
c	*O*	*G*	*A*	*F*	*E*	*Ă*	*Â*	*B*	*D*	*H*	*Î*	*J*	*K*	*L*	*M*	*N*	*Q*	*S*	*Ș*	*Ț*	*U*	*V*	*W*	*X*	*Y*	*Z*	*C*	*R*	*I*	*P*	*T*

Table 2. *Caesar cipher with keys $k_1 = 5$ and k_2 = criptografie (Romanian alphabet, n = 31)*

2.5. Input validation

```
def read_key():
    while True:
        value = input(f"Key (integer from 1 to {N - 1}): ").strip()
        if value.isascii() and value.isdigit() and 1 <= int(value) <= N - 1:
            return int(value)
        print(f"Invalid key '{value}'. "
              f"The key must be an integer from 1 to {N - 1} inclusive.")


def read_keyword():
    while True:
        value = input(f"Keyword (at least {MIN_KEYWORD_LENGTH} letters): ").strip()
        value = unicodedata.normalize("NFC", value)
        invalid = [ch for ch in value if ch not in UPPER_OF]
        if invalid:
            print(f"Invalid character '{invalid[0]}' in keyword. "
                  f"Use only {LETTERS}, no spaces.")
        elif len(value) < MIN_KEYWORD_LENGTH:
            print(f"Keyword too short ({len(value)} letters). "
                  f"It must have at least {MIN_KEYWORD_LENGTH} letters.")
        else:
            return "".join(UPPER_OF[ch] for ch in value)
```

```
def read_text(prompt):
    while True:
        try:
            text = prepare_text(input(prompt))
        except ValueError as error:
            print(f"Invalid character '{error.args[0]}'. Use only {LETTERS} and spaces.")
            continue
        if text:
            return text
        print("The text must contain at least one letter.")
```

Each value is read inside a `while True` loop and returned only when it is valid; otherwise a message is printed and the value is asked for again. `read_key` accepts only an integer written with the digits 0–9 between 1 and 30 inclusive (so `0`, `31`, `-3`, `2.5` or words are rejected) and the error message states the allowed range. `read_keyword` checks that the keyword has only Romanian letters (no spaces, digits or symbols), shows the first wrong character, checks that it has at least 7 letters and returns it in uppercase. `read_text` uses `prepare_text` and prints the rejected character, and it also rejects an empty text (for example only spaces).

2.6. Main program

```
    operation = read_option("Operation (e - encrypt, d - decrypt): ", ["e", "d"])
    key = read_key()
    alphabet = ALPHABET
    if task == "2":
        keyword = read_keyword()
        alphabet = permuted_alphabet(keyword)
        print("Permuted alphabet:", " ".join(alphabet))
    if operation == "e":
        text = read_text("Message: ")
        print("Prepared message:", text)
        print("Ciphertext:", encrypt(text, key, alphabet))
    else:
        text = read_text("Ciphertext: ")
        print("Decrypted message:", decrypt(text, key, alphabet))
```

This is the part of `main()` after the task is chosen. For Task 1.2 the keyword is read, the permuted alphabet is built and printed, and then it replaces the normal alphabet. After that the same `encrypt` / `decrypt` calls are used for both tasks.

2.7. Testing

To make sure the functions are correct I also wrote a few unit tests in `test_caesar.py` (module `unittest`). Because `encrypt`, `decrypt` and `permuted_alphabet` take the alphabet as a parameter, I could check them with the English examples from the lab description: CIFRULCEZAR with $k = 3$ gives FLIUXOFHCDU, BRUTEFORCEATTACK with $k = 17$ gives SILKVWFITVRKKRTB, and with the keyword CRYPTOGRAPHY the permuted alphabet is CRYPTOGAHBDEFIJKLMNQSUVWXZ and CIFRULCEZAR becomes PLKTXQPJYDT. The tests also check the Romanian codes from Table 1, the cedilla letters, and that decrypting an encrypted Romanian text gives back the same text for all 30 keys in both versions. All 5 tests pass.

3. Results

Below are screenshots of the program running. For both tasks I used the same message, **"Securitatea informației"**, and the same numeric key $k = 5$, so the two results can be compared.

3.1. Task 1.1 – encryption and decryption

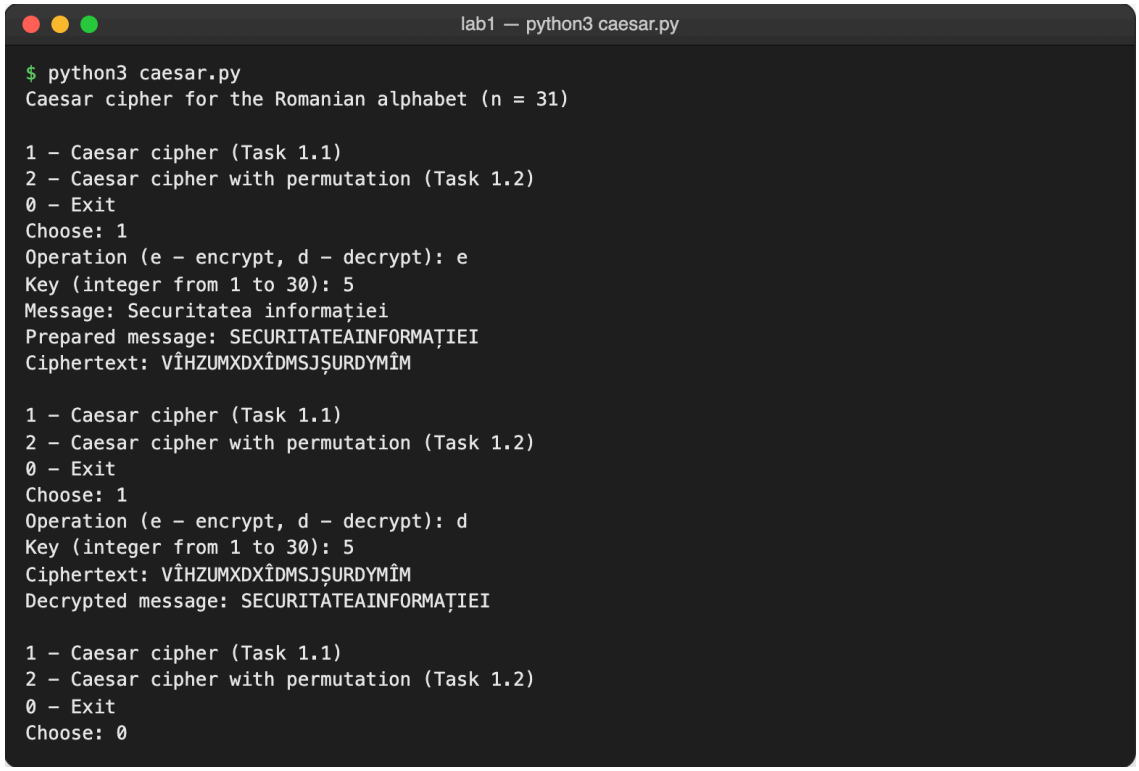


Figure 1. *Task 1.1: encryption and decryption with k = 5*

The message is converted to uppercase and the space is removed, giving `SECURITATEAINFORMAȚIEI`. With $k = 5$ the ciphertext is `VÎHZUMXDXÎDMSJȘURDYMÎM`. Decrypting it with the same key gives back the prepared message. I also checked the first letters by hand with Table 1: S (21) → 26 = V, E (6) → 11 = Î, C (4) → 9 = H.

3.2. Task 1.2 – encryption and decryption

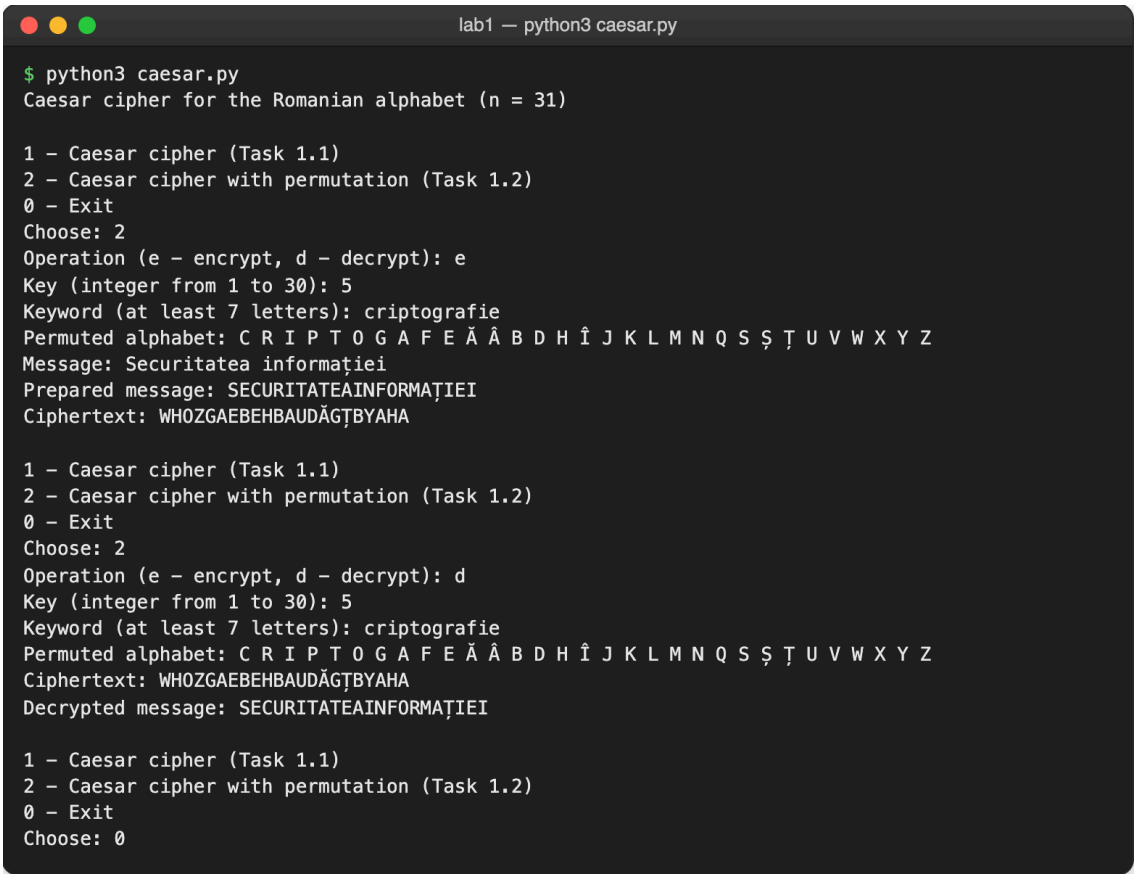


Figure 2. *Task 1.2: encryption and decryption with $k_1 = 5$ and k_2 = criptografie*

The program prints the permuted alphabet, which is the same as in Table 2. The same message and the same numeric key now give a completely different ciphertext, `WHOZGAEBEHBAUDĂGȚBYAHA`, and decryption with both keys restores the message.

3.3. Invalid input – Task 1.1

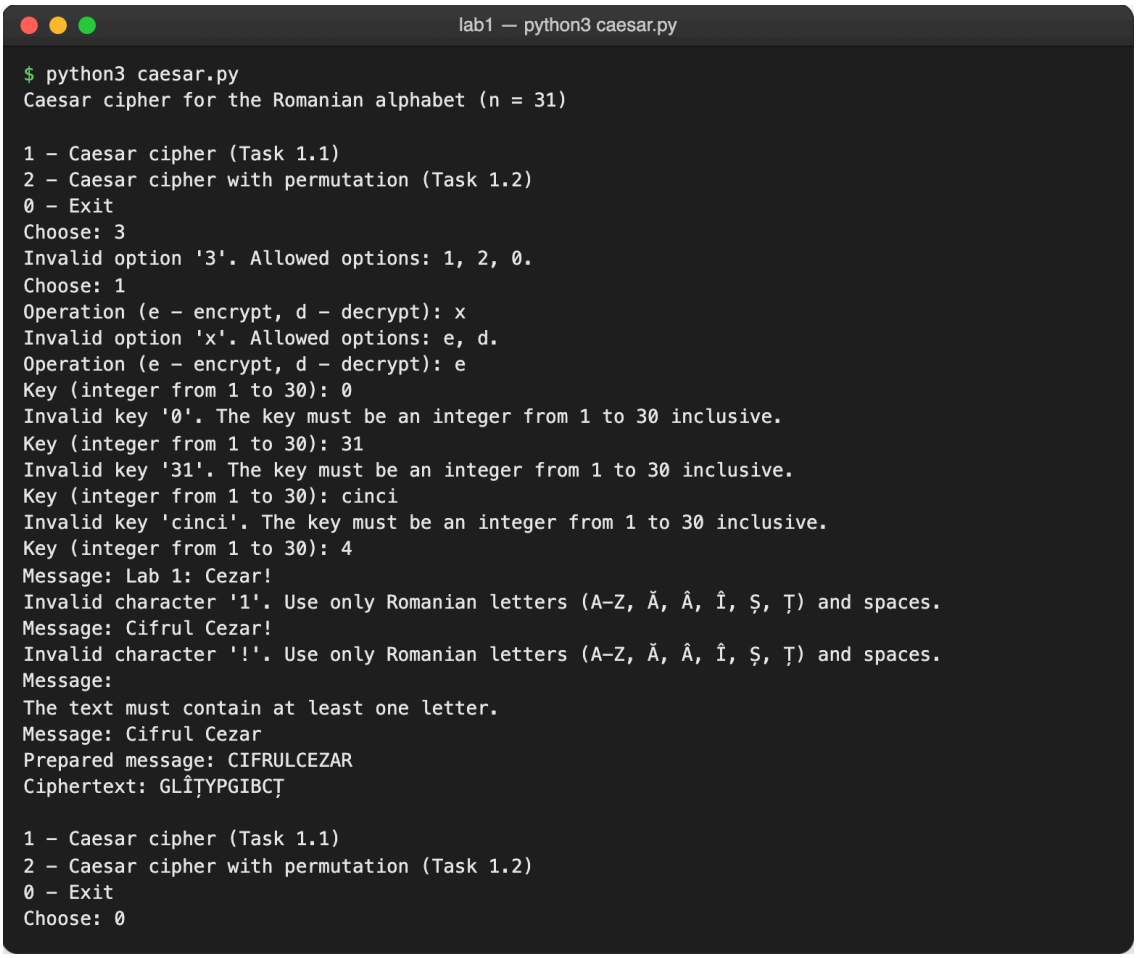


Figure 3. *Messages for an invalid menu option, invalid keys and invalid characters (Task 1.1)*

Here I tried a wrong menu option (3) and operation (x), the keys 0, 31 and “cinci”, then the texts “Lab 1: Cezar!” (rejected because of the digit 1) and “Cifrul Cezar!” (rejected because of !), and a text made only of spaces. Each time the program says what was wrong, gives the allowed values and asks again. The last attempt is valid: CIFRULCEZAR with $k = 4$ gives GLÎȚYPGIBCȚ, where Z → B shows the wrap-around to the beginning of the alphabet.

3.4. Invalid input – Task 1.2

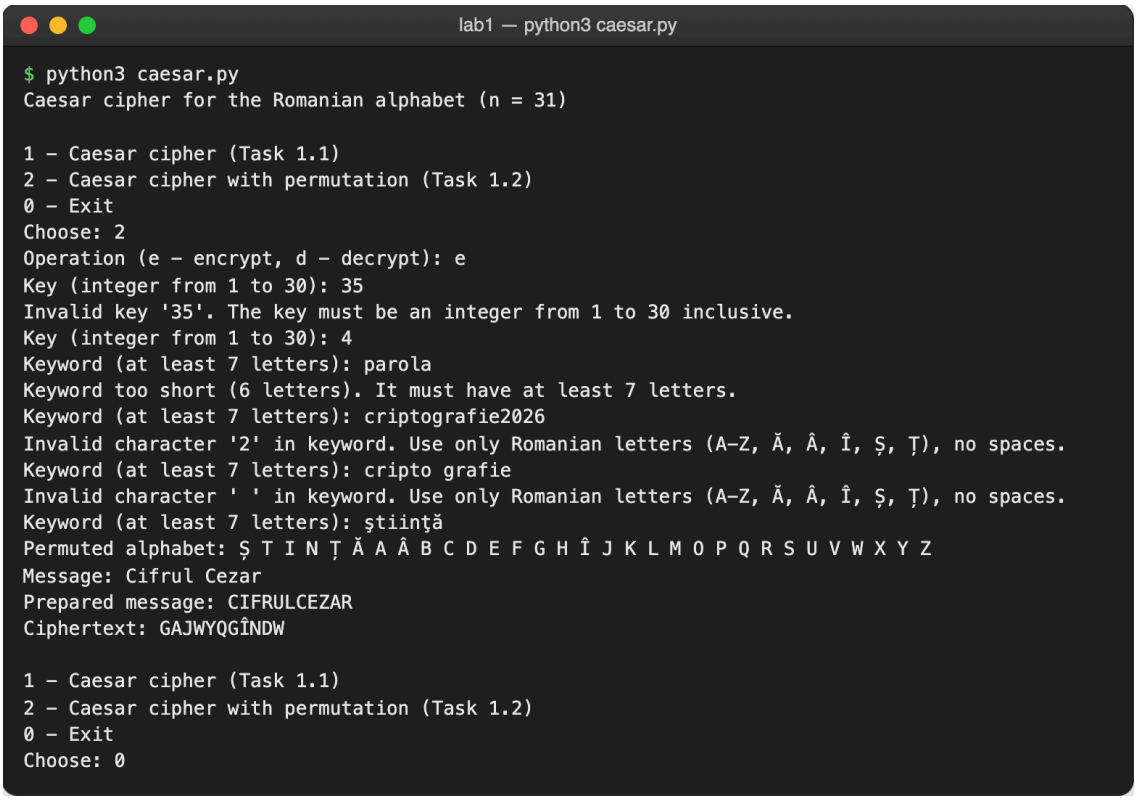


Figure 4. *Messages for an invalid key and invalid keywords (Task 1.2)*

The key 35 is rejected, the keyword “parola” is too short (6 letters), “criptografie2026” contains a digit and “cripto grafie” contains a space. The keyword “ştiinţă” was typed with the **cedilla** letters ş and ţ; it is accepted and converted to ȘTIINȚĂ with comma below, which can be seen in the printed permuted alphabet.

3.5. Brute-force check of Task 1.1

To see in practice how weak the simple Caesar cipher is, I decrypted the ciphertext from Figure 1 with all 30 keys using the `decrypt` function of the program (Table 3). Only $k = 5$ gives readable Romanian text, so the message is found immediately even without knowing the key.

1	UIGYȚLWCWICLRÎSȚQCXLIL	16	IVȚLHZJRJVRZDWEHCRKZVZ
2	ȚHFXTKVBVHBKQIRTPBWKHK	17	HUTKGYÎQÎUQYCVDGBQJYUY
3	TGEWȘJUÂUGÂJPHQȘOÂVJGJ	18	GȚȘJFXIPIȚPXBUCFÂPÎXȚX
4	ȘFDVSÎȚĂȚFĂÎOGPSNĂUÎFÎ	19	FTSÎEWHOHTOWÂȚBEĂOIWTW
5	SECURITATEAINFORMAȚIEI	20	EȘRIDVGNGȘNVĂTÂDANHVȘV
6	RDBȚQHȘZȘDZHMENQLZTHDH	21	DSQHCUFMFSMUAȘĂCZMGUSU
7	QCÂTPGSYSCYGLDMPKYȘGCG	22	CRPGBȚELERLȚZSABYLFȚRȚ
8	PBĂȘOFRXRBXFKCLOJXSFBF	23	BQOFÂTDKDQKTYRZÂXKETQT
9	OÂASNEQWQÂWEJBKNÎWREÂE	24	ÂPNEĂȘCJCPJȘXQYĂWJDȘPȘ
10	NĂZRMDPVPĂVDÎÂJMIVQDĂD	25	ĂOMDASBÎBOÎSWPXAVÎCSOS
11	MAYQLCOUOAUCIĂÎLHUPCAC	26	ANLCZRÂIÂNIRVOWZUIBRNR
12	LZXPKBNȚNZȚBHAIKGȚOBZB	27	ZMKBYQĂHĂMHQUNVYȚHÂQMQ
13	KYWOJÂMTMYTÂGZHJFTNÂYÂ	28	YLJÂXPAGALGPȚMUXTGĂPLP
14	JXVNÎĂLȘLXȘĂFYGÎEȘMĂXĂ	29	XKÎĂWOZFZKFOTLȚWȘFAOKO
15	ÎWUMIAKSKWSAEXFIDSLAWA	30	WJIAVNYEYJENȘKTVSEZNJN

Table 3. *Decryption of VÎHZUMXDXÎDMSJȘURDYMÎM with every key k = 1…30*

4. Conclusions

In this laboratory work I implemented the Caesar cipher and the Caesar cipher with a keyword permutation for the Romanian alphabet with 31 letters, using only the letter encoding from Table 1 and with validation of every input.

The classic Caesar cipher is very easy to implement, but it has practically no security. For the Romanian alphabet it has only 30 keys, and as Table 3 shows, only one key gives meaningful text, so anyone can break it in seconds by trying all of them, even by hand.

The version with a permutation increases the key space to $31! \times 30 \approx 2.47 \cdot 10^{35}$, so brute force is no longer realistic. However, the cipher is still a monoalphabetic substitution and does not hide the letter frequencies. This is visible even in my short example: in the plaintext SECURITATEAINFORMAȚIEI the letter I appears 4 times and E and A appear 3 times each, and in the ciphertext WHOZGAEBEHBAUDĂGȚBYAHA the letter A appears 4 times and H and B 3 times each. On a longer text, frequency analysis would recover the substitution, so a large key space alone does not make a cipher secure.

During this work I also learned some practical things: how to compute the modulo correctly for negative values, that the Romanian letters Ș and Ț exist in two Unicode forms (comma below and cedilla) and the program has to accept both, that Unicode text should be normalized before it is processed, and how to write input loops that clearly explain to the user what values are allowed.

5. Source code

The full source code (`caesar.py`, `test_caesar.py`) is available in the GitHub repository:

https://github.com/httpcatalin/cs-labs (folder `lab1`)